

Contents

1	Note on Lab 8	2
2	Exploring and preparing the dataset	2
3	Training and evaluating the model	3
4	Logistic regression model	5

1 Note on Lab 8

This lab solution only involves the use of the code taught in the tutorial, i.e. with the `knn()` function from `class`. Again, I would emphasise and encourage you to use the `train()` function from `caret`, which is much more robust in the training process. It also simplifies your training process.

Again, take note on the use of the `preProcess` parameter in the `train()` function, if you have already normalised your data.

2 Exploring and preparing the dataset

The data is stored in a `.csv` file, which we read in. We can see that there are 400 observations of 4 variables, and `admit` is our outcome variable of interest. The scales of all the features are also extremely different.

```
grad <- read.csv('gradadmit.csv', header=T)
str(grad)
```

```
## 'data.frame':    400 obs. of  4 variables:
## $ admit: int  0 1 1 1 0 1 1 0 1 0 ...
## $ gre  : int  380 660 800 640 520 760 560 400 540 700 ...
## $ gpa  : num  3.61 3.67 4 3.19 2.93 3 2.98 3.08 3.39 3.92 ...
## $ rank : int  3 3 1 4 4 2 1 2 3 2 ...
```

```
summary(grad)
```

##	admit	gre	gpa	rank
## Min.	:0.0000	Min. :220.0	Min. :2.260	Min. :1.000
## 1st Qu.:	:0.0000	1st Qu.:520.0	1st Qu.:3.130	1st Qu.:2.000
## Median :	:0.0000	Median :580.0	Median :3.395	Median :2.000
## Mean	:0.3175	Mean :587.7	Mean :3.390	Mean :2.485
## 3rd Qu.:	:1.0000	3rd Qu.:660.0	3rd Qu.:3.670	3rd Qu.:3.000
## Max.	:1.0000	Max. :800.0	Max. :4.000	Max. :4.000

We convert `admit` into a factor and perform a min-max normalisation on the predictors.

```
grad$admit <- factor(grad$admit)

nor <- function(x) { (x - min(x)) / (max(x) - min(x)) }
grad.nor <- grad
grad.nor[2:4] <- sapply(grad.nor[2:4], nor)
str(grad.nor)
```

```
## 'data.frame':    400 obs. of  4 variables:
## $ admit: Factor w/ 2 levels "0","1": 1 2 2 2 1 2 2 1 2 1 ...
## $ gre  : num  0.276 0.759 1 0.724 0.517 ...
## $ gpa   : num  0.776 0.81 1 0.534 0.385 ...
## $ rank  : num  0.667 0.667 0 1 1 ...
```

3 Training and evaluating the model

To train the model, we split the data into a training and test dataset as usual. In this case, the data is split into the predictors (`train.x` and `test.x`) and the outcome (`train.y` and `test.y`) for easier input into `knn()`. We see that the $k = 1$ achieves the best accuracy.

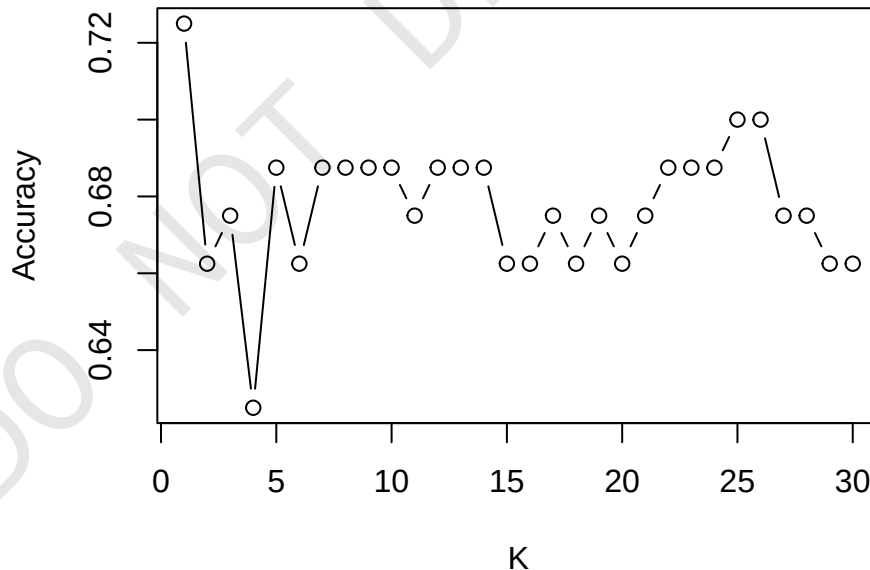
```
set.seed(100)
train.idx <- sample(nrow(grad.nor), nrow(grad.nor)*0.8)
train.x <- grad.nor[train.idx, 2:4]
train.y <- grad.nor[train.idx, 1]
test.x <- grad.nor[-train.idx, 2:4]
test.y <- grad.nor[-train.idx, 1]

library(class)
accuracy <- rep(0, 30)
for (i in 1:30) {
  set.seed(101)
  knn.i <- knn(train.x, test.x, cl=train.y, k=i)
  accuracy[i] <- mean(knn.i == test.y)
  cat("k =", i, "accuracy =", accuracy[i], "\n")
}
```

```
## k = 1 accuracy = 0.725
## k = 2 accuracy = 0.6625
## k = 3 accuracy = 0.675
## k = 4 accuracy = 0.625
## k = 5 accuracy = 0.6875
## k = 6 accuracy = 0.6625
## k = 7 accuracy = 0.6875
## k = 8 accuracy = 0.6875
## k = 9 accuracy = 0.6875
## k = 10 accuracy = 0.6875
## k = 11 accuracy = 0.675
## k = 12 accuracy = 0.6875
## k = 13 accuracy = 0.6875
## k = 14 accuracy = 0.6875
## k = 15 accuracy = 0.6625
```

```
## k = 16 accuracy = 0.6625
## k = 17 accuracy = 0.675
## k = 18 accuracy = 0.6625
## k = 19 accuracy = 0.675
## k = 20 accuracy = 0.6625
## k = 21 accuracy = 0.675
## k = 22 accuracy = 0.6875
## k = 23 accuracy = 0.6875
## k = 24 accuracy = 0.6875
## k = 25 accuracy = 0.7
## k = 26 accuracy = 0.7
## k = 27 accuracy = 0.675
## k = 28 accuracy = 0.675
## k = 29 accuracy = 0.6625
## k = 30 accuracy = 0.6625
```

```
plot(accuracy, type="b", xlab="K", ylab="Accuracy")
```



```
which.max(accuracy)
```

```
## [1] 1
```

Evaluating this model with the confusion matrix, we see this kNN model has an accuracy of 72.5%, a false negative rate of 40% (10/25), and a false positive rate of 21.8% (12/55).

```
set.seed(101)
yhat.knn <- knn(train.x, test.x, cl=train.y, k=1)
mean(yhat.knn == test.y)
```

```
## [1] 0.725
```

```
table(predicted=yhat.knn, observed=test.y)
```

```
##           observed
## predicted  0    1
##           0 43 10
##           1 12 15
```

4 Logistic regression model

We train and evaluate a logistic regression model on the same data. Since `train.x` and `test.x` do not contain the outcome variable `admit` (from how I split the data earlier), they have to be added back. You may also recreate the datasets, indexing the dataset with `train.idx`.

```
train.data <- cbind(train.x, admit=train.y)
test.data <- cbind(test.x, admit=test.y)
```

The training of the model goes as usual. Note that the interpretation of the coefficients is extremely different, as the training and test data we used was scaled. A unit change now represents the entire range of values of one predictor. For example, a unit change in `gre` takes us from 0 to 1, which corresponds to an unscaled `gre` of 220 to 800. This scaling in the predictors, however, does not affect our final results.

The logistic model achieves an accuracy of 68.8%, a false negative rate of 76%, and a false positive rate of 10.9%. How can you bring down this false negative rate? Will there be any trade-offs?

```
m.logistic <- glm(admit ~ ., train.data, family="binomial")
summary(m.logistic)
```

```
##
## Call:
## glm(formula = admit ~ ., family = "binomial", data = train.data)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -1.5737      0.5603  -2.808  0.00498 **
```

```
## gre          1.0851      0.7108      1.526      0.12689
## gpa          1.4679      0.6252      2.348      0.01888 *
## rank        -1.8531      0.4320     -4.289     1.79e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 400.59  on 319  degrees of freedom
## Residual deviance: 366.39  on 316  degrees of freedom
## AIC: 374.39
##
## Number of Fisher Scoring iterations: 4
```

```
y <- test.data$admit
yhat.p <- predict(m.logistic, test.data, type="response")
yhat <- factor(ifelse(yhat.p > 0.5, 1, 0))
mean(yhat == y)
```

```
## [1] 0.6875
```

```
table(predicted=yhat, observed=y)
```

```
##           observed
## predicted  0    1
##           0 49 19
##           1  6  6
```